INSTRUCTOR COPY Answers to the all-in-one booklet. Do not hand out.

Image Analysis Challenge: answers

All numbers are from `cow-demo.jpg` (320 by 320) and were recomputed with the code in `code/solutions`. Counting is from 0.

Part 1 explore a picture

Task 1 `pixels.shape` is `(320, 320, 3)`: 320 rows, 320 columns, 3 numbers per pixel (red, green, blue). Picture has $320 \times 320 = 102\,400$ pixels.

Task 2 `pixels[0, 0]` is `[16 0 0]` (almost black). `pixels[100, 200]` is `[94 50 37]`. The biggest is the first (red), so it is a reddish brown.

Task 3 Any 100 by 100 slice, for example `pixels[100:200, 100:200]`. The cow's head is a slice that needs about a 100 by 100 window around the upper middle of the cow; accept any sensible slice. Common mistake: swapping rows and columns.

Try it Yellow: high red, high green, low blue, e.g. (255, 255, 0). Grey: three equal numbers, e.g. (128, 128, 128).

Task 4 Green is `pixels[:, :, 1]` and blue is `pixels[:, :, 2]`. The wall is brightest in red; the cow is dark in all three channels.

Task 5 No red looks blue-green (cyan). No green looks pink-purple (magenta). No blue looks yellow-brown. Only blue: set channels 0 and 1 to 0. Prediction is their own; accept any reasoned guess.

Task 6 The negative of `[16 0 0]` is `[239 255 255]`. The black cow becomes pale. Each grey pixel has one number.

Task 7 `grey.mean()` is 81.45. Pixels darker than 50: 28501. Brighter than 100: 34315. Fraction $34315/102400 = 0.335$, about a third, so less than half.

Task 8 Grey histogram has a hump of dark pixels on the left (cow and street) and a second hump on the right (the wall). Red: tall bar middle-right (the wall). Green and blue: tallest on the left.

Stretch Mirror is `pixels[:, ::-1]`. Upside down is `pixels[::-1]`. $200+60 = 260$ wraps round to 4 in a byte (0 to 255), so bright pixels turn dark.

Part 2 the puzzle

Steps 2 to 4 Test values: `total_red("cow-demo.jpg")` prints `10218857`; `letter_for("cow-demo.jpg")` prints `Z` ($10218857 \bmod 26 = 25$). Model answer:

```
def total_red(path):
    img = Image.open(path)
    pixels = np.array(img)
    red = pixels[:, :, 0]
    total = red.sum()
    return total


def letter_for(path):
    total = total_red(path)
    position = total % 26
    letter = ALPHABET[position]
    return letter
```

```
def decode_message():
    message = ""
    for number in range(1, 38):
        letter = letter_for(image_path(number))
        message += letter
    return message
```

Stretch Green total of the demo photo is 7707223 and blue is 6674243. Different channel gives a different (nonsense) message. If a total goes up by 1 the next letter in the alphabet appears; up by 26 gives the same letter. `total_channel(path, channel)` is the same as `total_red` with `pixels[:, :, channel]`.